C Programming Language Review

Reminders

- C Review

- Quiz due Sunday
  - **On this weeks materials**

- Demo due Friday
  - **On interrupts!**
  - **We will be discussing them on Wednesday**
  - **Step through provided program**
    - Try stepping into function call

Definition and Declaration

- C Terminology
  - **Definition**
    - Place where a variable is created, assigned storage
  - **Declaration**
    - Nature of variable state is declared but no storage assigned
    - All variables must be declared before used
  - **Example**

```
int func_dec(void);     /* function prototype */
extern my_data;         /* declaration */
int def_func(void)
{
  float f_var;          /* a definition */
  int counter;          /* another definition */
  int rand_num(void);   /* declare (but not define)
  return(0);
}
```

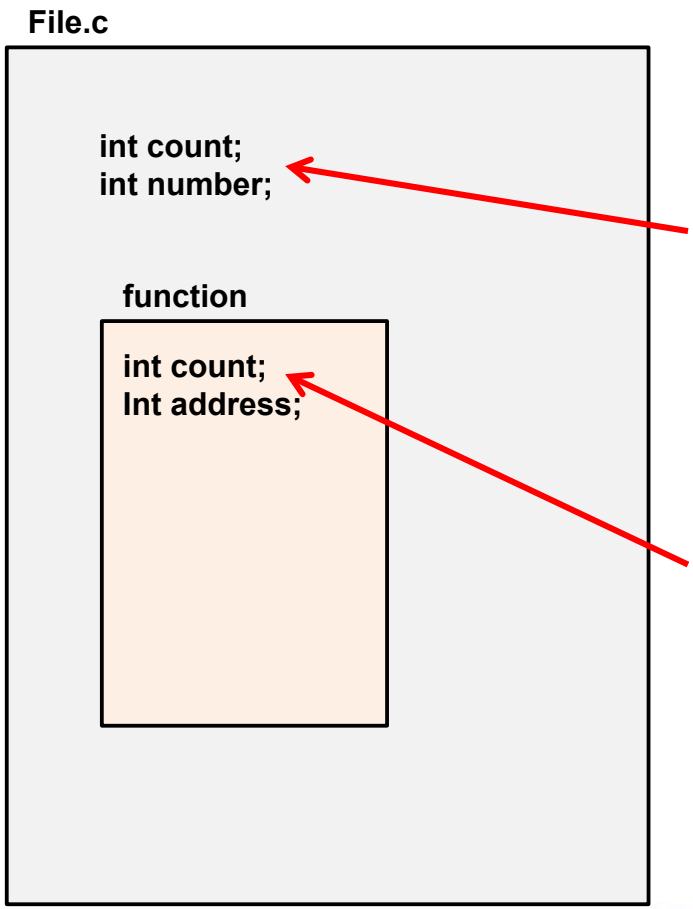
Definition and Declaration

- Distinction between the two is important
  - **You must declare the names of things before you can use them**
    - Exceptions: functions that return int and names of labels
      - ***But… you should declare them***
      - ***Otherwise you may get compiler warnings***
  - **When you use *declaration,* you introduce the name and type of something but no storage is allocated**
  - **When you use *definition*, you introduce the name and type along with allocating the space it will use**

Scope and Visibility

- C Terminology
  - **Scope**
    - Part of program within which a name can be used
    - Largest region in which the name is valid
    - Interrelated with the storage duration of identified object
      - ***Length of time an object remains in region of storage***
  - **Visibility**
    - Region of program text from which the object associated with the name can be legally accessed
    - Scope can exceed visibility, but visibility cannot exceed scope
      - ***Ex: Object hidden by duplicate identifier in inner declarative region***

Scope and Visibility



- Scope of ***count*** and ***number*** is entire file
- Variable resides in memory for entire file

- Scope of ***count*** and ***address*** is function
- Function variable ***count*** blocks visibility of original ***count*** variable
- Function variable only reside in memory while in function

Data Types

- Data Types refer to system used for declaring variables or functions
- Define how much memory is required for variables
- Classification of Data Types
  - Basic Types
    - ***Arithmetic types: Integer, char, and floating point***
  - Enumerated Types
    - ***Arithmetic types with restricted values***
  - Void Type
    - ***Special type that has no value***
  - Derived Types
    - ***Pointers, Arrays, unions, etc***

Data Types

- Basic Data Types
  - **char**
    - Data type that represents a single byte (8-bits)
    - Signed value: -128 to 127
    - Unsigned value: 0 to 255
  - **int**
    - An integer reflects the natural size of processor
    - Signed & Unsigned value: range depends on size
    - Compiler is allowed to choose size based on hardware
      - ***shorts & ints are at least 16-bits but shorts cannot be larger than int***
      - ***long must be at least 32-bits***
    - short int and long int
      - ***The word "int" typically omitted***
      - ***Intent is to provide different size integers***

Data Types

- Basic Data Types
  - **float**
    - Single-precision floating point
    - Requires 4 bytes
    - Range: 1.2E-38 to 3.4E38
    - 6 decimals of precision
  - **double**
    - Double-precision floating point
    - Requires 8 bytes
    - 15 decimals of precision
  - **long double**
    - Requires 10 bytes

Nios II Data Types

– *C/C++ data types for the Nios II processor*

Type	Size (Bytes)	Representation
char, signed char	1	two's complement (ASCII)
unsigned char	1	binary (ASCII)
short, signed short	2	two's complement
unsigned short	2	binary
int, signed int	4	two's complement
unsigned int	4	binary
long, signed long	4	two's complement
unsigned long	4	binary
float	4	IEEE
double	8	IEEE
pointer	4	binary
long long	8	two's complement
unsigned long long	8	binary

Typedef

- Typedef
  - **Used to create custom data types names**
  - **Names used with typedef are not new data types**
    - Synonyms for data types it represents
  - **Object defined with typedef identifier are exactly the same as if the object were defined by explicitly with the identifier**
  - **Can be used to reinforce concept of hardware & memory**
  - **Ex:**

```
/* Unsigned integer definitions (32bit, 16bit, 8bit) follow... */
typedef unsigned int        uint32;
typedef unsigned short      uint16;
typedef unsigned char       uint8;

/* Signed integer definitions (32bit, 16bit, 8bit) follow... */
typedef int                 sint32;
typedef short               sint16;
typedef char                sint8;
```

Preprocessor

- Preprocessor
  - **Part of compiler**
  - **Processes file before compilation**
  - **Works on file line-by-line**
  - **Perform simple textual manipulation before passing modified text to compiler**
    - Ex: All occurrence of <***name***> replaced with <***value***>
  - **Only works on lines of text**
    - Any line that starts with **#** is a preprocessor directive

Preprocessor

- Preprocessor Statements
  - **File Inclusion - Header Files**
    - Command: **#include**
    - Causes the line containing it to be replaced completely by the contents of another file
    - General practice to define all public variables and functions in *.h
    - If filename is in quotes then search begins where source files are stored
    - If filename is in < >then search uses standard list of system directories

Preprocessor

- Preprocessor Statements
  - **File Inclusion - Header Files**
    - When header files include each other, you get problems where things are defined multiple times
      - ***To prevent problems, you can use a compiler directive checks whether a token is defined***
    - Ex:

```
#ifndef MY_HEADER_H_
#define MY_HEADER_H_
    enter your header stuff here
#endif /* end MY_HEADER_H_ */
```

    - Creates token named MY_HEADER_H_ the first time this file is included
      - ***If the file is included again, compiler will see the token defined and it will not include the file***
    - The token can be any name but you want it unique

Preprocessor

- Preprocessor Statements
  - **Symbolic Constants**
    - Command: **#define**
    - Used to define symbolic names or constants
      - ***Poor programming to embed "magic" numbers into code***
    - Format
      - `#define <name> <replacement value>`
      - ***Name of symbolic constant is typically all UPPERCASE***

Preprocessor

- Preprocessor Statements
  - **Conditional Inclusion**
    - Command: **#if**
    - Provides means to control the preprocessor with conditional statements
    - **#if** provides means to evaluate a constant expression
      - ***Ex:***

```
#if debug_enabled
 // include this debug code
#else
 // include non-debug code
#endif
```

Preprocessor

- Preprocessor Statements
  - **Conditional Inclusion**
    - #ifdef **symbol**
      - ***Check to see if symbol is defined***
      - ***Value does not matter***
    - #if defined(**symbol**)
      - ***Check to see if symbol is defined but value does not matter***
      - ***In simplest form it is the same as #ifdef***
      - ***Allows several things to be checked at once***
      -
```
#if defined(FOO) || defined(BAR)
  // do some stuff
#endif
```
    - #ifndef **symbol**
      - ***Check to see if symbol is NOT defined***

Constants

- Constants
  - **Unsigned integers**
    - Add suffix “u” for unsigned
    - Ex: 12345u
    - Ex: 12345ul → unsigned long
  - **Hexadecimal**
    - Prefix number for “0x”
    - Ex: 0x12C
    - Ex: 0x12Cu → unsigned value of 0x12C
  - **No binary representation**
    - A number with a leading zero will be considered octal!

C Programming Language Review

- Constants
  - **Character Constants**
    - An integer written as one character in single quotes
    - Ex: ‘O’
      - ***Value is number value of machine character set***
      - ***ASCII value of ‘O’ is 48***
  - **Escape Sequences**
    - Character Constants with **** added
    - Examples of useful escape characters
      - **\n ➔ New Line**
      - **\b ➔ Backspace character**
      - **\t ➔ Tab character**
      - **\r ➔ Return character**

Qualifiers

- Type Qualifiers
  - **Refine the declaration of a variable, a function, or parameter**
    - Provides additional information about the data type
  - **Type specifiers and qualifiers can be written in any order**
    - <qualifier> <data type> variable_name
    - <data type> <qualifier> variable_name

Qualifiers

- Type Qualifiers
  - **CONST**
    - CONST keyword tells compiler data will not be written to
      - ***Makes a variable read-only.***
    - Data typically gets allocated to const section instead of the .bss section
    - Example:
      - ***Can use pi in expressions but not change value***

```
const double pi = 3.14159265358979323846;
```

    - Similar to #define

```
#define PI 3.14159265358979323846
```

    - CONST creates a read-only variable
      - ***Can create pointer to pi but NOT to PI***

Qualifiers

- Type Qualifiers
  - **Volatile**
    - Declares an item whose value can legitimately change beyond control of program
      - ***Ex: User I/O and peripheral registers, ISRs***
    - Forces the suppression of optimization that could otherwise occur
      - ***Prevent removal of redundant references***
      - ***Prevents the caching of lvalue - each operation must be performed***
    - May be used to instruct compiler to bypass cache
      - ***Hardware dependent (Nios – Yes; Nios II – No)***
    - Example:

```
volatile int *push_button = PB_BASE_ADDRESS;
```

Qualifiers

- Type Qualifiers
  - **What happens if I don't use volatile?**
    - The code might work fine until interrupts are turned on
    - The driver may be inconsistent and return erroneous data, leading to long periods of debugging
    - Turning on compiler optimizations causes stable code base to fail
    - Code might work until a new task or thread is added to the application which then causing it to crash and burn for no apparent reason

Qualifiers

- Type Qualifier
  - **Restrict**
    - Can only be applied to a pointer
      - ***A pointer is the address of a location in memory.***
      - ***More than one pointer can access the same chunk of memory and modify it during the course of a program***
    - Restrict declares that the pointer it is qualifying is the only direct or non-direct way of accessing the object pointed to by pointer
      - ***Allows compiler addition optimization opportunities***
    - Not very common anymore
      - ***Compilers are more sophisticated***
    - Example : Loop unrolling
      - ***Restrict both source and destination pointers in a loop***

Storage Classes

- Storage Class
  - **Every C variable has a storage class and a scope**
  - **The storage class determines the part of memory where storage is allocated for an object and how long the storage allocation continues to exist**
  - **C has four storage classes**
    - auto
    - register
    - extern
    - static

Storage Classes

- Storage Class: auto
  - **Default for variables declared inside a block (i.e.: function)**
    - Objects are local to a block
    - Remember main () is a function
  - **auto specifier explicitly used to define storage**
  - **Created on entering block and discarded on exit**
    - Variables are created on stack
  - **Value is indeterminate when created**
  - **Typically not used…. Because it is the default!**
  - **Example:**

```
auto int count = 0;
```

Storage Classes

- Storage Class: register
  - **Tells compiler variable should be in processor register**
  - **Compiler is not required to honor this request**
  - **Limited by underlying hardware**
  - **Variables are automatic if not placed in register**
  - **Ideal for heavily used variables**
    - Attempt to enhance performance by minimizing access time
  - **Value is indeterminate when created**

  - **Typically not used…. Because compilers now-a-days make better decisions**
  - **Example:**

```
register int count = 0;
```

Storage Classes

- Storage Class: static
  - static **specifier explicitly used to define storage**
    - Mechanism in C to enforce information hiding
  - **Can be applied to any data type**
  - **Tells the compiler to make the variable or function limited in scope while allowing it to persist throughout the life of the program**
  - **Hides variables from the rest of the program to prevent inadvertent access**
  - **Variable in a block is initialized only one time, prior to program execution**
  - **Value is zero when created**
  - **Example:**

```
static int count = 0;
```

Storage Classes

- Storage Class: static

```
uint8_t Var1 = 0;

void Test_Function(void)
{
    uint8_t Var2 = 0;

    Var1++;
    Var2++;
}
```

```
uint8_t Var1 = 0;

void Test_Function(void)
{
    static uint8_t Var2 = 0;

    Var1++;
    Var2++;
}
```

- **Var2 is reset to 0 every time function is called**
- **Var2 only set to 0 during program initialization and then value is retained for each subsequent call**

Storage Classes

- Storage Class: extern
  - **extern specifier explicitly used to not allocate storage**
    - Allows a variable defined in one file to be declared in others
  - **Memory is allocated for an extern variable (via the global variable) before main begins and freed when the program terminates**
  - **Scope is from point it declared until end of file**
  - **Must be defined exactly once as a global variable in another file**
  - **Example:**
    - `extern int count;`

Storage Classes

- Storage Class: extern
  - **Globally Accessible Variables**
    - Advantages
      - ***Can be used to communicate data between files***
      - ***Avoids need of passing parameters between functions***
    - Disadvantages
      - ***Leads to programs with data connections that are not obvious***
        - » Change program in one spot effects unrelated area
      - ***Destroys the abstraction and isolation of functions***
      - ***Prevents code from being reentrant***
        - » Reentrant ➔ Re-entered code while it is already running
      - ***Nightmare to debug***

Functions

- Functions
  - **Provide way to encapsulate computation**
  - **Provide a means to do work w/o the user knowing details**
    - Hides details of implementation
  - **Declaration**
    - Ex: `int power (int base, int exp);`
      - ***Function named power with two parameters***
      - ***Function returns integer***
  - **Parameters**
    - Variable names in parentheses list of declaration
    - Local to the function
  - **Arguments**
    - Variables in the function call

Functions

- Functions
  - **Return value**
    - Not required – use void
      - ***Omitting return is the same as falling off the end of function***
    - Last statement is the ***return*** statement
      - ***May or may not have expression to return***
      - ***Typical Return***
        - » Zero implies normal termination
        - » Non zero implies error
  - **Function prototypes**
    - Declaration of function to define parameter and return types
    - Parameter and return type must match function definition
    - Parameter names are optional

Functions

- Functions
  - **Scope**
    - Globally accessible unless restricted with ***static***
  - **Main is a function**
    - May return a value

Functions

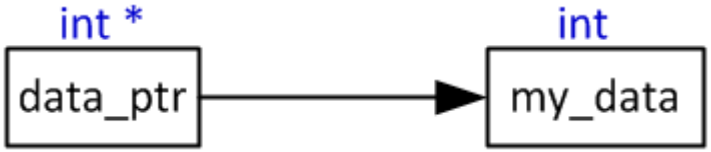
- Passing Arguments to Function
  - **Call by Value**
    - All function arguments are passed by value
      - ***Means copy of arguments passed into function***
      - ***Changes to arguments are not reflected in original variable***
      - ***Function cannot alter arguments***
    - Function Call Format: `function (arg1, arg2)`
    - Danger in exceeding stack limit by copying larger data structures to stack
      - ***Very hard to debug***
    - Nios II Compiler supports run-time stack memory overflow checks while debugging
      - ***See BSP Property***

Functions

- Passing Arguments to Function
  - **Call by Reference**
    - Function is passed the address of the argument
      - ***Function can alter argument's values***
    - Function must declare parameters as pointers
      - ***function (int * arg1, int * arg2)***
    - Function Call Format: `function (&Var1, &Var2)`
    - Pass larger structures to functions as pointers
      - ***Reduces stack usage***

Pointers

- Using Pointers
  - **A pointer is a variable that contains a memory address**
    - Memory address could be another variable or I/O device
    - Ex:



  - **Define pointer variables by preceding name with asterisk**
    - Ex: `int *ptr;`
      - ***ptr is the name of the variable***
      - ***"*" informs the compiler we want pointer variable***
        - » Compiler set aside enough bytes to store address
      - ***int says pointer variable stores address of an integer***
  - **To access data pointed to by pointer use dereferencing**
    - Ex: `*ptr = 7;`
      - ***Copies 7 to the address pointed to by ptr***

Pointers

- Using Pointers
  - **Memory Map Example 1**
    - uint16 Var1 = 0x000E;
    - uint16 Var2 = 0x0003;
    - uint16* PtrInt = &Var1;

Address	Value	Variable
0x0100	0x000E	Var1
0x0102	0x0003	Var2
0x0104	0x00000100	PtrInt

    - Var1 has address of 0x0100 and value 0x000E
    - Var2 has address of 0x0102 and value 0x0003
    - The pointer has the address of Var1 so the memory location 0x00000100 is accessed with either Var1 or *PtrInt

*** is used to dereference a pointer**

& is used to get the address of a variable

Pointers

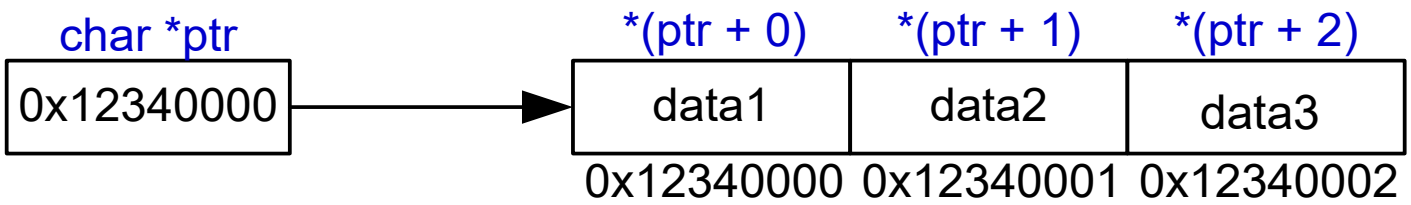
- Using Pointers
  - **Memory Map Example 2**
    - uint16 Var1 = 0x000E;
    - uint16 Var2 = 0x0003;
    - uint16* PtrInt = Var1;

Address	Value	Variable
0x0100	0x000E	Var1
0x0102	0x0003	Var2
0x0104	0x0000000E	PtrInt

    - Var1 and Var2 are the same as the last example
    - PtrInt is assigned the value of Var1
    - This will likely result in a compiler warning
      - ***Data type mismatch***
        - » uint16 and uint16*

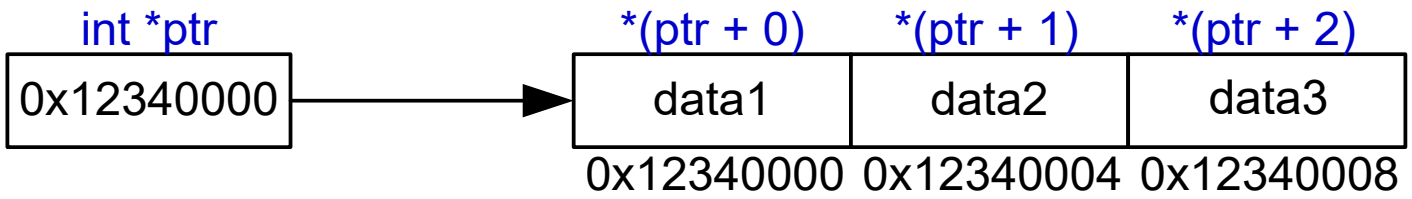
Pointers

- Using Pointers
  - **Incrementing pointer is based on data type**
    - Ex: `char *ptr;`



      - ***Address increments by 1 since each data is 1 byte***

    - Ex: `int *ptr;`



      - ***Address increments by 4 since each data is 4 byte***

Bitwise Operators

- Bitwise Operators
  - **Allow for direct bit manipulation**
  - **Examples:**
    - & ➔ bitwise AND
    - | ➔ bitwise OR
    - ^ ➔ bitwise XOR
    - << ➔ left shift
      - ***Remember the vacated bit are fill zero***
    - >> ➔ right shift
      - ***Vacated bits are filled 0 for unsigned numbers***
      - ***Vacated bit of signed fill value is machine dependent***
        - » Either 0 or sign bit
    - ~ ➔ one's complement

Bitwise Operators

- Bitwise Operators
  - **Used to manipulate bits within register w/o affecting other bits**
  - **Typically done as part of read-modify-write operation**
    - Read current value of register
    - Modify register value (may use bitwise operation)
    - Write new register value
  - **To set a bit in register use bit-wise OR**
    - Example to set bit 4

```
new_value = old_value | 0x0010
```

  - **To clear a bit in register use bit-wise AND**
    - Example to clear bit 4

```
new_value = old_value & ~0x0010
```